

Guión de presentación

TP3 — Perceptrones

Orden cronológico de la presentación

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Cómo está organizado este guion

Este documento sigue el **orden real de la presentación**, slide por slide. / Cada bloque tiene un orador asignado — los oradores se cruzan varias veces. / Si lo lees de arriba hacia abajo en el momento de presentar, sabés exactamente dónde estás y quién habla.

Secuencia (36 slides totales, ~ 24 minutos):

#	Bloque	Tema	Slides	Tiempo
1	A — Mateo	Apertura + Ej 1 setup + análisis dataset	1–5	3:30
2	B — Katia	Lineal vs no-lineal + threshold	6–8	3:30
3	C — Nicolás	Setup Ej 2 + fases + sweeps arch/opt	9–13	3:30
4	D — Tomás	Decisiones de diseño (activación + viz + init + opt)	14–17	4:00
5	E — Nicolás	Learning rate + convergencia/bias	18–21	3:15
6	F — Tomás	Curvas + matriz + distribución + cachetazo	22–26	3:30
7	G — Mateo	Hipótesis Ej 3 + el dominó +10 puntos	28–29	1:55
8	H — Katia	Matriz ganador + tabla resultados + comparación	30–32	1:50
9	I — Mateo	Curvas del ganador + qué ayudó	33–34	2:00
10	J — Katia	Por qué no 98 %	35	1:15

Las slides 9 (dividir Ej 2), 27 (dividir Ej 3) y 36 (Gracias) las cubre el orador adyacente, sin contar tiempo aparte.

Totales por orador:

Persona	Bloques	# slides	Tiempo
Mateo	A + G + I	9	7:25
Katia	B + H + J	7	6:35
Nicolás	C + E	9	6:45
Tomás	D + F	9	7:30

Reglas no negociables:

- Se lee como uno habla — español rioplatense, fluido, sin tecnicismos innecesarios.
- No se dice “distribution shift” ni “cambio de población”. La narrativa nueva es: “el

train no contiene la clase 8” y la clase 5 está sub-representada. Es un problema de **cobertura**, no de shift.

- **No se dice “Pack C”**. Hablamos de “regularización”.
- **No se dice “vanishing gradient”** a secas — se explica con la palabra “saturar” y se aclara la consecuencia.

Convenciones: **0:30** = dónde tendrías que estar dentro del bloque · **[Slide N]** = cambiar de slide · / pausa breve · // pausa más larga · **→ Mateo→Katia** = pasarle el bastón.

Índice

1. Bloque A / Mateo / Apertura + Ej 1 setup	1
2. Bloque B / Katia / Lineal vs no-lineal + threshold	2
3. Bloque C / Nicolás / Setup Ej 2 + sweeps arch/opt	3
4. Bloque D / Tomás / Decisiones de diseño	4
5. Bloque E / Nicolás / Learning rate + convergencia/bias	5
6. Bloque F / Tomás / Curvas, matriz, distribución y cachetazo	6
7. Bloque G / Mateo / Hipótesis Ej 3 + el dominó	7
8. Bloque H / Katia / Matriz, tabla y comparación visual	7
9. Bloque I / Mateo / Curvas del ganador + qué ayudó	8
10. Bloque J / Katia / Por qué no 98 %	8
Apéndice — preguntas anticipadas	9

Bloque A / Mateo / Apertura + Ej 1 setup

/ / 3:30

Foco: arrancar la presentación + setup del Ejercicio 1 + análisis del dataset (las tres reglas + cómo la sigmoide actúa como un OR aproximado).

[Slide 1, portada] [Bloque A]

Buenas. Trabajo Práctico tres, perceptrones. / Somos cuatro: Katia, Nicolás, Tomás y yo. Nos vamos turnando en bloques cortos, así que vamos a estar yendo y viniendo. / Al final dejamos lugar para preguntas.

0:25

[Slide 2, divider Ej 1]

Arrancamos con el Ejercicio uno, que es lo que llaman *knowledge distillation* sobre un dataset de fraude. / El dos y el tres son de clasificación de dígitos manuscritos, los vemos después.

0:45

[Slide 3, Ej 1 — el problema]

A ver, qué nos pide el Ejercicio uno. // Tenemos un BigModel que ya viene entrenado y clasifica fraude. Para nosotros es una caja negra: lo único que vemos es su salida, una probabilidad continua entre cero y uno. / La idea es entrenar un modelo chico — le decimos *TinyModel* — que sea un perceptrón simple y replique esa salida.

// El dataset tiene nueve features de la transacción: monto, duración de sesión, edad de la cuenta, esas cosas. / Y trae también una columna que se llama `flagged_fraud`, que es la etiqueta binaria de fraude real. *El enunciado dice claramente que esa columna no se puede usar para entrenar*: la usamos solo después, como criterio de evaluación con métricas operativas — precision, recall, F1.

// Como la salida tiene que estar entre cero y uno, la activación final del perceptrón es **sigmoide**.

1:30

[Slide 4, Ej 1 — análisis del dataset: tres reglas]

Antes de entrenar nada, miramos el dataset y nos encontramos con algo bastante llamativo. // Hay tres umbrales bien marcados, en tres features distintas, que separan las clases casi perfecto. / Si el monto es mayor a 500, o la cantidad comprada es 10 o más, o los items vistos antes de la compra son 15 o más / *cualquiera de las tres*, casi seguro es fraude. / Lo ven en la fórmula del cuadrito de arriba.

// El plot que está abajo muestra la *forma de escalón* que tienen esas tres features: hay un umbral donde la probabilidad de fraude pega un salto. / Cualquiera de las tres alcanza por sí sola para detectar el grueso de los fraudes.

2:20

[Slide 5, plain con escalones + insight sigmoide]

Y acá está el insight clave del Ejercicio uno. // Las tres features con umbral tienen forma de S marginal: piso bajo, después una subida rápida, y después se quedan en el techo. / Y todas apuntan al mismo lado: si la feature está alta, la probabilidad de fraude es alta.

// ¿Qué hace la sigmoide del perceptrón con eso? / Toma las nueve features, las combina linealmente con sus pesos en un *score*, y le aplica **una sola S** en la dirección que apunta el vector de pesos. / Si los pesos de las tres features clave son positivos — como deberían ser — ese score termina actuando como un **OR aproximado**: alcanza con que alguna de las tres dispare para que el score se vaya alto y la sigmoide saturate en uno.

// Y por contraste, el modelo lineal no puede curvar. / Termina prediciendo valores fuera de cero a uno en el siete por ciento del dataset. Esa es la pista de por qué la sigmoide le va a ganar al lineal: forma de S coherente con la naturaleza del target.

→ **Mateo→Katia** Le paso a Katia para que muestre los números.

Bloque B / Katia / Lineal vs no-lineal + threshold

/ / 3:30

Foco: cerrar el Ejercicio 1 con la decisión de threshold (lo más de negocio del TP).

[Slide 6, Ej 1 — Lineal vs No-Linear] [Bloque B]

Buenas, soy Katia. / Mateo dejó la pista de que la sigmoide debería ganarle al lineal. La respuesta corta: sí, y por bastante.

// Acá, en la tabla del medio, comparamos las dos versiones. Mismo dataset, mismo K-fold=5 estratificado, mismas features. / Lo único que cambia entre los dos es la sigmoide. // El *lineal* —que es Adaline puro— da MSE de 0.0266. / El *no lineal*: **0.0110. 59% menos de error.**

// La razón es justo la que terminó de explicar Mateo: el target es una probabilidad acotada entre cero y uno. / La sigmoide *automáticamente* mapea su salida a ese rango. El lineal puede tirarte valores fuera de cero a uno, y el MSE te castiga fuerte cada vez que cae fuera del dominio. / La activación final tiene que ser coherente con la naturaleza del target.

1:10

[Slide 7, Ej 1 — pruebas sobre threshold]

Ahora viene la parte que más nos hizo pensar. // Si tomamos el modelo no lineal con threshold 0.5 —el default obvio para una sigmoide— las métricas operativas dan mal. / Precision 33 %, recall 100 %. Tira casi todo a fraude.

// A primera vista parece un modelo malísimo. Pero la cuestión es que el modelo no es el problema — *el threshold no está calibrado al modelo*. / ¿Por qué? Porque las salidas de la sigmoide se concentran en la parte de arriba. La mediana de los scores de las muestras de fraude está en 0.99. Y los *no fraudes* no se van a 0.05 como uno esperaría — quedan en 0.4 o 0.6. / La red aprendió un *ranking* muy bueno, pero no un calibrado perfecto.

// La solución fue barrer el threshold de cero a uno con paso 0.01. En el plot ven precision en azul, recall en verde, F1 en negro. / El óptimo por F1 está en **threshold 0.89**: precision 89, recall 86, F1 87, accuracy 97 %.

2:10

[Slide 8, Ej 1 — recomendación de umbral]

Y la última pregunta del Ejercicio uno: *qué umbral le recomendamos al cliente.* // La respuesta es: depende. Depende del costo asimétrico que tenga el cliente entre falso positivo y falso negativo.

// La tabla muestra cuatro puntos. / Threshold 0.89 maximiza F1: agarra el 86 % de los fraudes con 96 falsos positivos. Esto sirve si auditar manualmente cada flagged es barato. / Threshold 0.95 baja a 35 falsos positivos pero pierde recall: solo recupera el 75 %. Esto sirve si los falsos positivos son caros para el negocio. / Y threshold 0.99 te da precision perfecta y cero falsos positivos — pero pierde la mitad de los fraudes reales. Esto es para cuando hay tolerancia cero a falsos positivos.

// El modelo entrega scores. Decidir el umbral es una decisión del negocio.

→ **Katia**→**Nicolás** Le paso a Nicolás para arrancar el Ejercicio dos.

Bloque C / Nicolás / Setup Ej 2 + sweeps arch/opt

/ / 3:30

Foco: arrancar el Ejercicio 2 (problema, fases, sweep arquitectura y optimizador). Tablas y gráficos hablan solos.

[Slide 9, divider Ej 2] [Bloque C]

Buenas, soy Nicolás. / Pasamos al Ejercicio dos.

0:15

[Slide 10, Ej 2 — el problema]

Clasificación multiclase de dígitos manuscritos del cero al nueve. // Cada muestra es una imagen de 28 por 28 píxeles, en escala de grises, aplanada en *un vector de 784 floats entre cero y uno.* / Train: 12.449 muestras. Test: 2.498.

// La arquitectura final, después de todos los sweeps que les voy a mostrar, es: / entrada de 784, capa oculta de 100 con ReLU, otra de 50 con ReLU, salida de 10 con softmax. / Y la regla del juego, en la caja roja: **el test se usa una sola vez, al final.** La búsqueda de hiperparámetros es solo sobre **digits.csv**.

1:00

[Slide 11, Ej 2 — fases de experimentación]

Para no probar al azar, dividimos la búsqueda en **dos fases**. // *Fase uno, exploratoria:* K-folds = 1, o sea un solo train-val 80-20. / Sweeps secuenciales: arquitectura, después optimizador, después learning rate, después batch size. / Cada sub-fase usa el ganador de la anterior, así reducimos un montón el espacio de búsqueda.

// *Fase dos, validación:* K-fold = 5. / Tomamos el config ganador de la primera fase y variamos un solo hiperparámetro a la vez para ver si las elecciones se sostienen con K-fold completo. / Y al final, un único *final eval* contra el test set — una sola vez, como dije recién.

1:40

[Slide 12, Ej 2 — sweep de arquitectura]

Empezamos por arquitectura. // Probamos cuatro variantes. / La fila de arriba, 128-64, dio 96.86 en validación. / Y la fila verde —ganadora— es 100-50 con 96.74. / La diferencia entre las dos es 0.12 puntos, con un desvío estándar de 0.4 / *están dentro del ruido:* es empate técnico. // Más abajo, 100 sola dio igual pero converge más lento, y 50 sola quedó subajustada.

// La decisión la tomamos por **simplicidad de la arquitectura:** cuando dos modelos rinden igual, te quedás con el más chico y rápido. / Es el principio que se llama parsimonia: no vale la pena el doble de parámetros por una décima que está dentro del error.

2:30

[Slide 13, Ej 2 — sweep de optimizador]

Sweep de optimizador. / **Adam** con learning rate 0.001: 96.74, *best epoch* 7. Ganador. / Momentum con $\beta = 0,9$: 96.34, converge más lento —*best epoch* 18. / SGD vanilla: 94.81, y la

realidad es que *no convergió en 50 epochs*, llegó al límite todavía mejorando. // Adam ganó por convergencia más rápida y resultado igual o mejor. Momentum quedó dentro del desvío, SGD sin aceleración no puede.

→ **Nicolás** → **Tomás** Le paso a Tomás para que cuente *por qué* elegimos cada cosa — las decisiones de diseño con backing teórico.

Bloque D / Tomás / Decisiones de diseño

/ / 4:00

Foco: las decisiones donde el Q&A puede ir más profundo (activación, init, optimizador). Incluye una slide de visualización ReLU/softmax.

[Slide 14, Función de activación] [Bloque D]

Buenas, soy Tomás. / Nicolás mostró *qué* elegimos en cada sweep, ahora me toca contar el *por qué*. // Tres decisiones de diseño que aplican a todos los ejercicios: activación, inicialización y optimizador. / El learning rate y la función de costo los retoma Nicolás cuando vuelva.

// Función de activación, caso por caso. / En el Ejercicio uno usamos **sigmoide** porque el target es una probabilidad acotada en cero a uno. / En Ejercicio dos y tres, las capas ocultas con **ReLU** y la salida con **softmax**. // ¿Por qué ReLU en las capas ocultas? Porque el gradiente es fuerte y constante en zona positiva. *No satura* como tanh o sigmoide. / ¿Qué quiere decir saturar? Sigmoide y tanh, en sus extremos, tienen pendiente cercana a cero. Cuando el gradiente vuelve hacia atrás por el backprop atravesando muchas capas con sigmoide, va multiplicando esos números chicos y termina llegando casi nulo a las primeras capas. La red deja de aprender. / ReLU evita ese problema porque en su zona activa la pendiente es exactamente uno: el gradiente fluye sin atenuarse.

// Y softmax en la salida es directo: para clasificar 10 clases, queremos una distribución de probabilidad sobre las diez. Softmax la devuelve normalizada.

1:30

[Slide 15, ReLU y Softmax — visualización]

Acá los tienen graficados. // A la izquierda, ReLU. / Es la función máximo entre cero y z : a partir de cero pasa por la diagonal, antes es plana. / La derivada es uno en zona positiva y cero en negativa. Esa es la propiedad clave que evita la saturación.

// A la derecha, softmax. / Toma un vector de números cualquiera y los transforma en una distribución de probabilidad. / El que tiene el score más alto se va cerca de uno; los demás se reparten el resto. La suma de los diez valores siempre da uno. / Por eso encaja perfecto con cross-entropy como función de costo: queremos que la salida *sea* una probabilidad sobre las diez clases.

2:15

[Slide 16, Inicialización de pesos]

Inicialización. // *La regla de la cátedra*, que sale del HTML que nos compartieron, es: *valores chicos, no cero, no muy grandes*. / ¿Por qué no cero? Por simetría. Si todas las neuronas arrancan iguales, hacen exactamente lo mismo en cada paso del gradiente y *nunca se diferencian*. / ¿Por qué chicos? Porque pesos grandes saturan las activaciones — la salida queda en zonas planas, el gradiente es casi cero, la red no aprende.

// Tres esquemas. **He**: distribución normal con desvío $\sqrt{2/\text{fan_in}}$, para capas con ReLU. La intuición es que ReLU descarta la mitad del input —todo lo negativo se va a cero—, así que doblamos la varianza para compensar. / **Xavier**: $\sqrt{1/\text{fan_in}}$, para tanh, sigmoide y softmax, que son simétricas alrededor de cero. / **Uniforme** entre menos 0.1 y más 0.1, para el perceptrón simple del Ejercicio uno donde no hay propagación profunda.

3:10

[Slide 17, Optimizador]

Optimizador. // *La clase introduce gradiente descendente clásico*; Momentum y Adam son extensiones modernas. Comparamos los tres. // **SGD vanilla**: la versión mínima, w menos eta por

gradiente. Funciona pero es lento. / **Momentum** con $\beta = 0,9$: acumula *velocidad* en la dirección del gradiente. Atraviesa valles sin oscilar. / **Adam**, con sus tres hiperparámetros default: combina *momentum* con *learning rate adaptativo por parámetro*.

// Los números que vio Nicolás: Adam 96.74 con best epoch 7. Momentum 96.34 — empate técnico pero converge más lento. SGD 94.81 — no convergió en 50 epochs. // Elegimos Adam por convergencia más rápida y resultado igual o mejor.

→ **Tomás→Nicolás** Le paso a Nicolás para learning rate y los detalles de convergencia y bias.

Bloque E / Nicolás / Learning rate + convergencia/bias

/ / 3:15

Foco: sweep de learning rate (con extremos catastróficos) y dos detalles de implementación (convergencia y bias).

[Slide 18, Learning rate — cuál y por qué] [Bloque E]

Vuelvo yo. // Tomás cubrió las decisiones de diseño; ahora me toca contar cómo encontramos el learning rate ganador. // *La recomendación de la clase es: probar varios órdenes de magnitud.* / Los riesgos son simples: si lo elegís muy grande, la red oscila o diverge; si lo elegís muy chico, no converge en tiempo razonable.

// La búsqueda fue en **dos pasos**. / Fase uno exploratoria, con cinco valores entre 10^{-4} y 10^{-2} . / Fase dos con esos mismos cinco más *extremos catastróficos* — 0.1 y 10 — para mapear el rango donde diverge.

0:50

[Slide 19, learning rate — Fase 1]

Fase uno con K-folds = 1. // La fila verde, learning rate 0.001: 96.74, best epoch 7. Ganador. / 0.0005 queda muy cerca pero converge más lento, best epoch 17. / 0.0001 no termina de converger en 50 epochs. / Y 0.005 con 0.01 hacen overshoot —best epoch 2 a 4, demasiado temprano.

// Tres criterios apuntan al mismo valor: máximo de validación, convergencia rápida, sin overshoot.

1:30

[Slide 20, learning rate — Fase 2]

Y la confirmación con K-fold=5. // Mismos cinco valores más los extremos. / 0.001: 96.22, ganador. / 0.0001: 3 veces más lento, val_acc apenas peor. / 0.01: overshoot. // Y los extremos: 0.1 cae a 29% — divergencia clara. / Y **learning rate igual a 10** deja a la red en 12.48 — accuracy random, equivalente a tirar una moneda con diez caras. El loss explotó a NaN antes de converger.

// Conclusión: **learning rate 10^{-3} confirmado por dos sweeps independientes.**

2:15

[Slide 21, convergencia y bias]

Y para terminar mi parte, dos detalles de implementación que no son hiperparámetros pero *el TP nos pide discutir*. // **Criterio de convergencia.** / En el Ejercicio uno usamos un *epsilon absoluto* sobre el MSE de train — la regla del apunte para perceptrón simple. / En Ejercicio dos y tres usamos **early stopping sobre el loss de validación** con patience de 10 epochs. / La razón: si miramos solo el train, no detectamos overfitting. Con validación, sí.

// **Cómo manejamos el bias.** / Truco del bias: a cada capa le agregamos una columna de unos al input antes del producto matricial, y los pesos pasan a tener shape **(n_out, n_in+1)**. / El bias queda como un peso más y se actualiza con la misma regla del gradiente.

→ **Nicolás→Tomás** Le paso a Tomás para las curvas, la matriz, la distribución de clases y el cachetazo del final eval.

Bloque F / Tomás / Curvas, matriz, distribución y cachetazo

/ / 3:30

Foco: la parte clave del Ejercicio 2. La narrativa nueva: **el cachetazo se explica porque el train no contiene la clase 8**. La slide 24 (distribución de clases) es la pieza clave.

Tono: narrativo. Construí el cachetazo con la pausa correcta antes de decir “86.30”.

[Slide 22, Ej 2 — curvas del modelo base] [Bloque F]

Vuelvo yo. // Estas son las curvas de aprendizaje del modelo base, evaluadas con K-fold=5. / Train y validación bajan parejas, convergencia limpia en alrededor de 5 epochs. / **Sin overfitting visible:** las dos curvas se mantienen pegadas hasta el final.

0:25

[Slide 23, Ej 2 — matriz del base (val K=5)]

Matriz de confusión sobre validación. // Diagonal limpia para 9 de las 10 clases. / La excepción —y esto es clave— es la **clase 8**: precision cero, recall cero. La red *nunca predice ocho*. / Eso ya nos dio la pista: `digits.csv` tiene cobertura insuficiente de la clase 8 — pocos ejemplos, o ejemplos poco representativos.

0:55

[Slide 24, Ej 2 — distribución de clases en train vs test]

Y acá está la pieza que termina de explicar todo. // Comparamos cuántos ejemplos hay de cada clase en el train y en el test. / Las nueve clases que no son la 5 ni la 8 tienen alrededor de 1500 muestras cada una en el train. // La clase 5 tiene **271** — está sub-representada. // Y la clase 8 tiene **cero**. / **El train no contiene un solo 8**. El modelo *literalmente nunca vio un 8* durante el entrenamiento.

// Por eso la columna 8 de la matriz colapsa: si nunca le mostraste un 8, no puede predecirlo.

// El test, en cambio, sí está balanceado: tiene unos 250 ejemplos por clase, ochos incluidos. / Eso significa que **243 muestras del test — los ochos — ya son error garantizado** para nuestro modelo del Ejercicio dos. / 243 sobre 2497 da casi exacto el 9.7 por ciento de error que vamos a ver en el cachetazo.

2:00

[Slide 25, Ej 2 — final eval: el cachetazo]

Y entonces hicimos el *final eval*. / Entrenamos con todo `digits.csv` y evaluamos una sola vez contra el test set, `digits_test.csv`. // Resultado.

// Validación K-fold=5: **96.22 %**. / Test sobre el set de prueba: **86.30 %**. // **10 puntos de drop**.

// Y la explicación está justo en lo que acabo de mostrar. / El K-fold daba 96 porque dentro de `digits.csv` no había clase 8 para evaluar. El modelo aprendió 9 de las 10 clases bien y el K-fold reportaba accuracy sobre esas 9. / Cuando llega el test con 243 ochos, los falla todos. / El drop sale prácticamente exacto: el 9.92 por ciento que cae es casi idéntico al 9.7 por ciento de ochos en el test.

3:00

[Slide 26, Ej 2 — matrices validación vs test]

Y las dos matrices lado a lado para verlo. // A la izquierda, la matriz de validación: la columna 8 está vacía. / A la derecha, la matriz sobre el test: aparece la fila 8 con ejemplos reales, y la red los confunde con todas las otras clases. / Errores distribuidos en la clase 8 —justamente la que nunca vio.

// La interpretación es entonces clara: el techo del Ejercicio dos no es problema del modelo, es problema de **cobertura**. Si querés que la red prediga una clase, necesitás mostrársela en el train. Esa es la motivación entera del Ejercicio tres.

→ **Tomás→Mateo** Le paso a Mateo, que cuenta cómo el Ejercicio tres confirmó esa hipótesis.

Bloque G / Mateo / Hipótesis Ej 3 + el dominó

/ / 1:55

Foco: el momento “mirá esto”. Más datos = +10 puntos en test, sin tocar el modelo. La clase 8 ahora se predice.

[Slide 27, divider Ej 3] / (Lo cubrís de paso, sin contar tiempo.)

[Slide 28, Ej 3 — hipótesis y plan] [Bloque G]

Vuelvo yo. // Tomás recién mostró que el techo del Ejercicio dos viene de la cobertura desigual de las clases 5 y 8 — sobre todo la 8 que faltaba entera. / Si esa es la hipótesis, lo primero que hay que probar es *más datos* que cubran lo que falta, no un modelo más grande.

// *Y el enunciado del Ejercicio tres nos da justo eso:* un dataset extra, `more_digits.csv`, con 15.742 muestras nuevas. / El plan fue en dos pasos. Uno, agregar el dataset extra sin tocar el modelo. Si llegábamos al 98 % con eso, listo. / Dos, si no alcanzaba, activar regularización — L2, dropout, augmentation gaussiana, learning rate scheduling — y ver qué combinaciones funcionaban.

0:50

[Slide 29, Ej 3 — el movimiento dominó]

Y este es el resultado del primer paso, que sinceramente nos sorprendió. // Pegamos los dos CSVs antes del split. **Cero cambios al modelo**, solo cambia un campo del config. // Miren los números.

// Validación K-fold=5: subió de 96.22 a 97.06. / Y el test, que es lo que importa: **de 86.30 a 96.36. // 10 puntos en test, sin tocar el modelo.**

// Y miren acá, en la caja verde: / macro F1 saltó de 0.85 a 0.95. / La clase 8 que estaba colapsada —justo la que faltaba en el train original— ahora la red la predice bien. / El dataset extra cubrió el agujero. Eso *confirma* la hipótesis: el problema era cobertura de clases, no capacidad del modelo.

→ **Mateo→Katia** Le paso a Katia para que muestre la matriz, la tabla y la comparación con el Ej 2.

Bloque H / Katia / Matriz, tabla y comparación visual

/ / 1:50

Foco: mostrar los resultados completos del Ej 3 (matriz ganador, tabla de regularización, comparación con Ej 2).

[Slide 30, Ej 3 — matriz del ganador] [Bloque H]

Vuelvo después del dominó de Mateo. // Esta es la matriz de confusión del modelo ganador del Ejercicio tres, evaluada sobre el test. / La diagonal está bastante más limpia que la del Ejercicio dos. / La clase 8 que en el dos colapsaba ahora se predice bien, y los errores que quedan se reparten entre varias clases.

0:35

[Slide 31, Ej 3 — resultados (tabla)]

Esta es la tabla completa de los configs que probamos. // Vamos por la columna de test, que es la que vale.

Base con datos extra, sin nada de regularización: 96.36. / **Más L2**: 96.56, sumamos 0.20. / **Dropout solo**: peor en test, perdió 0.08. / **L2 más dropout combinados**: peor todavía, perdió 0.32. // Y la fila resaltada en verde, **L2 más augmentation gaussiana con sigma 0.05**: **96.88 %**. 0.52 puntos sobre el baseline. / Ese es nuestro ganador. / Las dos filas siguientes muestran que combinar todo o agrandar la red empeoraba.

1:05

[\[Slide 32, Ej 3 — comparación visual\]](#)

Y este es el plot que compara directamente el Ejercicio dos con el ganador del tres, sobre el mismo test. // Las cinco métricas, lado a lado. La línea roja punteada es el target del 98 %. / Lo que se ve es que la mayor parte del salto vino del dataset extra — ya las barras grandes están al nivel del ganador. La regularización aporta el último empujón.

→ [Katia](#)→[Mateo](#) Mateo vuelve para las curvas y el balance.

Bloque I / [Mateo](#) / Curvas del ganador + qué ayudó

/ / 2:00

Foco: cierre técnico antes de las conclusiones. Curvas del ganador y balance de qué movió la aguja.

[\[Slide 33, Ej 3 — curvas del modelo ganador\]](#) [Bloque I]

Vuelvo para mostrar las curvas y el balance. // Estas son las curvas de aprendizaje del modelo ganador — L2 más augmentation gaussiana, sobre el dataset extendido. / La convergencia es un toque más lenta que el modelo base del Ejercicio dos: *best epoch* alrededor de 10 en lugar de 5, lo cual tiene sentido porque la regularización ralentiza el ajuste. / Pero lo importante es que **train y validación bajan parejas**: no hay overfitting.

0:35

[\[Slide 34, Ej 3 — qué ayudó y qué no\]](#)

Y este slide es el resumen. // **Lo que ayudó:** / más datos, 10 puntos — de lejos lo más importante. / L2, 0.20 puntos sólidos. / Augmentation gaussiana, 0.32 puntos *adicionales* sobre L2 — las dos juntas suman.

// **Lo que no ayudó:** / Dropout. Gana en validación pero pierde en test. / Reduce varianza dentro del K-fold, pero no aporta donde el problema venía siendo la cobertura desigual de las clases 5 y 8. / La arquitectura *wider*, 200-100 neuronas: mejor en validación, peor en test. Más capacidad memoriza el train mejor pero no generaliza. Y eso *descarta* que el problema sea capacidad. / Y lo último, combinar todo a la vez — L2, dropout y augmentation juntos — nos dio peor que solamente L2 con augmentation. Cuando exageras la regularización, las técnicas se pelean entre ellas.

→ [Mateo](#)→[Katia](#) Katia cierra con por qué nos quedamos en 96.88.

Bloque J / [Katia](#) / Por qué no 98 %

/ / 1:15

Foco: cierre honesto del TP. Carisma para el final.

[\[Slide 35, Ej 3 — por qué no llegamos al 98\]](#) [Bloque J]

Cierro yo. // La pregunta incómoda. ¿Por qué no llegamos al 98 %? Mejor que tenemos: 96.88, faltan 1.12 puntos.

// La hipótesis principal es la siguiente. / El augmentation gaussiano que implementamos simula *ruido píxel a píxel*. Pero la diferencia que castiga al test parece más *geométrica*: distintas personas escriben los dígitos con rotaciones, traslaciones y grosores de trazo distintos. / Y acá viene el detalle importante: *en la clase de regularización vimos cuatro formas de augmentation*: ruido gaussiano, rotaciones, traslaciones y cambios de escala. **Solo implementamos la primera.** / Las otras tres son justo las que atacarían el tipo de variación que falta cubrir.

// Y agrandar el modelo —como mostró Mateo con *wider*— empeoraba. / O sea que **no es un problema de modelo, es un problema de cobertura**. // El siguiente paso natural es implementar augmentation geométrica. Quedó como trabajo a futuro.

[Slide 36, Gracias] Gracias. / Preguntas.

Apéndice — preguntas anticipadas y a quién derivar

¿Por qué eligieron threshold 0.89? (Katia)

Porque maximiza F1 en el sweep. Si el cliente prefiere precisión sobre recall, threshold mayor o igual a 0.95. Es decisión de negocio sobre el costo asimétrico, no del modelo.

¿Cuántas muestras tiene el dataset de fraude? ¿Cuál es la prevalencia? (Katia)

Aproximadamente 9 mil muestras totales. La clase positiva pesa alrededor del 12 %.

¿Qué es Adam? ¿No se ve en otra clase? (Tomás)

Adam combina dos ideas: *momentum* (promedio del gradiente) y *RMSProp* (learning rate adaptativo por parámetro, según el promedio del gradiente al cuadrado). *El PDF de optimizadores de la cátedra lo recomienda* con los defaults del paper original: $\alpha = 10^{-3}$, $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\varepsilon = 10^{-8}$.

¿Por qué He para ReLU y Xavier para tanh? (Tomás)

La varianza de las activaciones depende de la activación. ReLU descarta la mitad del input —todo lo negativo se va a cero—, así que para mantener la varianza balanceada por capa hay que multiplicar el desvío por raíz de dos. Xavier asume activaciones simétricas alrededor de cero (tanh, sigmoide).

¿Por qué dropout no funcionó en el Ejercicio tres? (Tomás)

Dropout reduce la varianza del modelo dentro del fold de cross-validation, pero el problema del Ejercicio dos no era varianza dentro del fold: era *cobertura desigual de las clases 5 y 8*. Una técnica que reduce varianza al fold no soluciona un agujero de cobertura. Por eso ganó en val pero no transfirió a test.

¿Por qué no llegan al 98 %? (Katia, con apoyo de Tomás)

El techo viene de la calidad del augmentation, no de la capacidad. *La clase de regularización menciona cuatro formas*: ruido gaussiano, rotaciones, traslaciones, cambios de escala. Solo implementamos la primera. Las otras tres son justo las que atacarían las variaciones geométricas (estilos de escritura) que quedan en el test.

¿Cómo eligieron la cantidad de capas y neuronas? (Nicolás, con apoyo de Tomás)

Por la curva U que vimos en la clase de regularización: capacidad muy baja subajusta, capacidad muy alta sobreajusta. Hicimos sweep alrededor del ganador y elegimos por simplicidad: cuando dos arquitecturas rinden igual, gana la más simple y rápida. *Wider* confirmó la curva U: más capacidad sin más datos $\Rightarrow$ peor en test.

¿Qué relación hay entre lo que hicieron y la clase de regularización? (Tomás)

La clase enseña cuatro técnicas: *early stopping* (slide 14), *augmentation* en cuatro formas (slide 18), *L2/weight decay* (slides 20-25) y *dropout* (slide 26 mencionado). Nosotros usamos las cuatro: early stopping en todo Ejercicio dos y tres; L2 con $\lambda = 10^{-4}$; augmentation gaussiana con $\sigma = 0,05$; y dropout probado pero descartado porque no transfirió. / Lo que faltó: las otras tres formas de augmentation del slide 18.

¿Cómo eligieron el tamaño de batch? (Nicolás)

Sweep en Fase uno con cuatro tamaños: 16, 32, 64, 128. Ganó 16: mismo val_acc que 64 pero *best epoch* en 3 versus 7 y total más rápido en wall clock.

¿Por qué `digits.csv` no tenía clase 8? ¿Es un bug del dataset? (Tomás)

No es un bug, es la composición del dataset que nos dieron. La clase 8 figura cero veces en `digits.csv` y está balanceada en `digits_test.csv`. Lo descubrimos al inspeccionar la matriz de confusión del Ejercicio dos. Justifica entera la motivación del Ejercicio tres con `more_digits.csv`.

Fin del guion. Slides en docs/slides_presentacion/slides_presentacion.pdf. Backing teórico para Q&A: docs/notas/.